

Manual Técnico

Sistema de Monitoreo Sísmico

Proyecto Sismógrafo | Versión MVP

Gustavo Andrés Angulo | Ana Maria Caviedes

1. Información General del Proyecto

Campo	Detalle
Nombre del proyecto	Sistema de Monitoreo Sísmico / Sismógrafo
Integrantes	Gustavo Andrés Angulo; Ana Maria Caviedes
Docente	No especificado en los archivos revisados
Tecnologías utilizadas	Node.js, Express, MySQL, JWT, Multer, ExcelJS, PDFKit, Python, Three.js, Chart.js, Leaflet, KaTeX y Arduino C++
Versión del sistema	MVP funcional

2. Descripción General

Aspecto	Descripción
Objetivo del sistema	Monitorear la actividad sísmica, mostrar telemetría en tiempo real, registrar lecturas, administrar usuarios y apoyar la sustentación con un módulo educativo.
Problema que resuelve	Centraliza la visualización de datos, la persistencia histórica y la administración de usuarios evitando depender exclusivamente de lecturas físicas durante la demostración.
Alcance del proyecto	Incluye la solución distribuida propuesta durante el semestre y el MVP funcional para demostración web, reportes y administración.

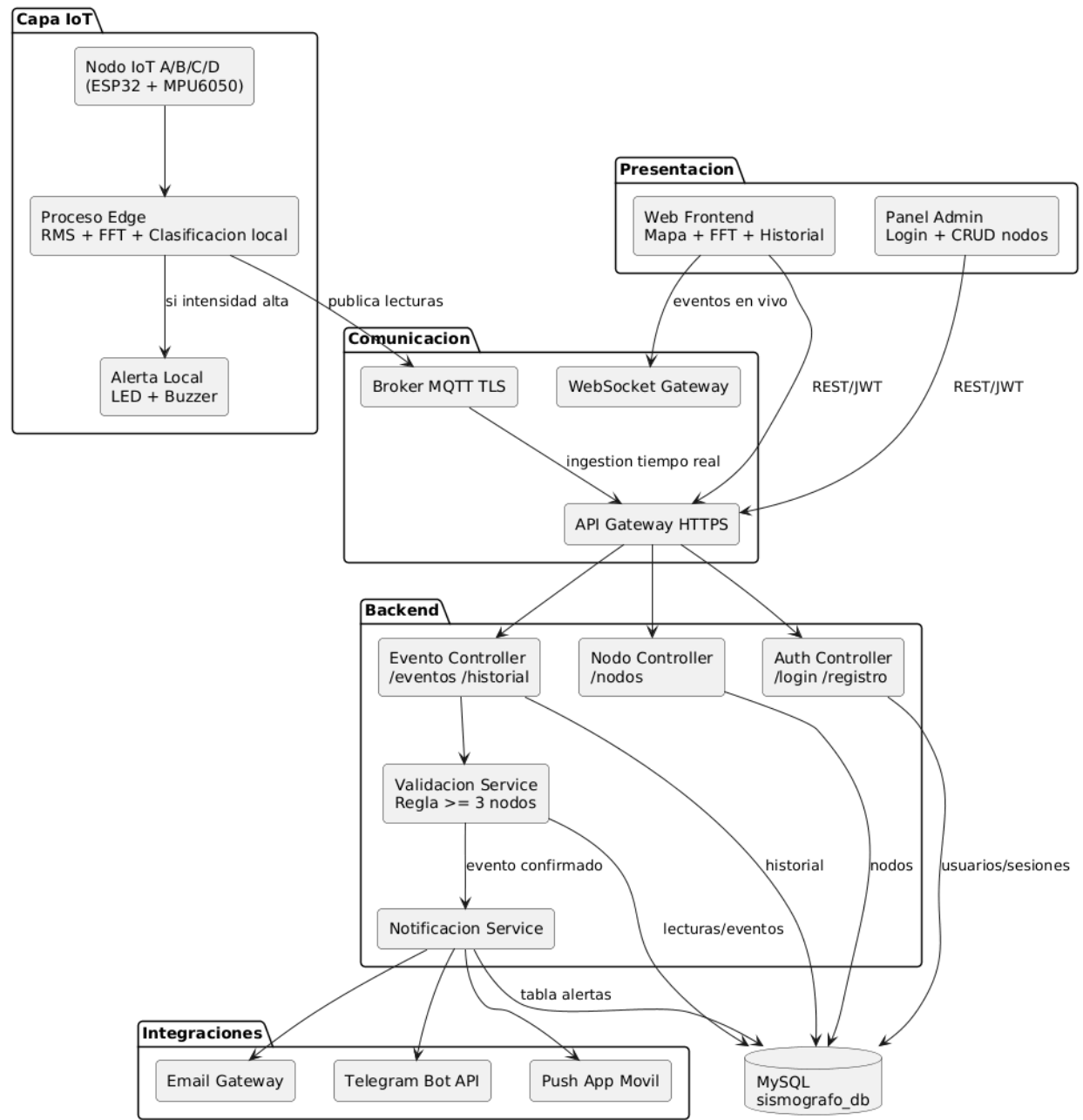
- La propuesta académica del curso contempla una solución distribuida con nodos IoT, validación multi-nodo, mapa en tiempo real, alertas y módulo educativo.
- El MVP funcional de esta entrega concentra la evidencia operativa en autenticación, panel web, reportes, historial, simulación y administración de usuarios.

3. Arquitectura del Sistema

- La arquitectura conceptual del proyecto está formada por nodos IoT, servidor central, plataforma web, sistema de notificaciones y base de datos relacional.
- Los nodos IoT del diseño del curso usan MPU6050 + ESP32/NodeMCU y transmiten lecturas por MQTT sobre TLS; el MVP operativo actual utiliza un puente serial y simulación web para la demo.
- La interfaz pública se sirve desde `index.html`, el panel administrativo desde `admin-panel.html` y el módulo de reportes desde `/reportes/`.
- El backend expone rutas REST para autenticación, administración, reportes, salud del sistema y telemetría del sismógrafo.

Componente	Función
Nodos IoT	Miden aceleraciones y transmiten lecturas al servidor.
Servidor central	Valida, agrega y distribuye la información sísmica.
Frontend público	Muestra dashboard, autenticación, modo educativo y acceso de invitado.
Panel admin	Permite iniciar sesión, administrar usuarios y generar reportes.
API backend	Valida credenciales, gestiona usuarios, reportes, archivos y eventos sísmicos.
Puente de hardware	Lee el sensor serial o levanta simulación si no existe el puerto físico.
Persistencia	Guarda usuarios, archivos, lecturas y eventos en MySQL.

Diagrama de Componentes - Arquitectura del Sistema Sismografo



- Diagrama seleccionado para sustentar: diagrama de componentes.
- Se eligió porque resume la estructura principal del sistema y cómo se conectan sus módulos.

4. Requerimientos Técnicos

Tipo	Requerimiento
Hardware requerido	PC de demostración, Arduino UNO o equivalente, sensor MPU6050, ESP32/NodeMCU para la propuesta distribuida, LEDs, buzzer, protoboar, cable USB.
Software requerido	Node.js, MySQL Server, Python 3, navegador moderno, Arduino IDE, cliente MQTT/broker y terminal PowerShell o equivalente.
Dependencias	cors, dotenv, exceljs, express, jsonwebtoken, multer, mysql2 y pdfkit; además Three.js, Chart.js, Leaflet, GSAP y KaTeX en el frontend.

5. Instalación y Configuración

1. Abrir la carpeta `C:\Users\Administrator\Desktop\Desarrollo\_Web\mi\_proyecto`.
2. Instalar dependencias con `npm install` si hace falta.
3. Configurar el archivo `.env` con el puerto, usuario, contraseña y nombre de la base de datos.
4. Si se requiere un usuario administrador inicial, ejecutar `npm run admin:bootstrap`.
5. Levantar el servidor con `npm start`.
6. Abrir `http://localhost:3000/`, `http://localhost:3000/admin-panel.html` y `http://localhost:3000/reportes/` para validar la interfaz pública, el panel de admin y el módulo de reportes.

Variable	Uso
PORT	Puerto HTTP del servidor.
DB_HOST	Host de MySQL.
DB_PORT	Puerto de MySQL.
DB_USER	Usuario de la base de datos.
DB_PASSWORD	Contraseña de MySQL.
DB_NAME	Nombre de la base de datos.
JWT_SECRET	Clave para firmar tokens JWT.
JWT_EXPIRES_IN	Tiempo de vida del token, opcional.
SISMO_SERIAL_PORT	Puerto serial del sensor, opcional.
SISMO_DB_PERSIST_INTERVAL_MS	Intervalo de persistencia del estado sísmico, opcional.

6. Base de Datos

- Modelo entidad relación: el documento completo del proyecto incluye el esquema distribuido con `nodos`, `lecturas\_nodo`, `eventos\_sismicos`, `evento\_nodo`, `usuarios`, `alertas` y `graficas\_fft`.
- Base operativa del MVP: además del modelo de referencia, el sistema actual persiste usuarios y eventos hardware en la tabla `eventos\_hardware\_sismo`.
- Scripts necesarios: el proyecto incluye el bootstrap del administrador (`scripts/bootstrap-admin.js`), la creación automática de la tabla de persistencia desde `server.js` y la base SQL de referencia `sismografo\_database.sql`.

Objeto	Campos o propósito
nodos	Identificador, nombre, latitud, longitud, estado, versión de firmware y fecha de instalación/conexión.
lecturas_nodo	Lecturas individuales con aceleraciones, RMS, frecuencia dominante, amplitud y clasificación local.
eventos_sismicos	Eventos validados por el servidor con intensidad, estado y coordenadas.
evento_nodo	Relación N:M entre eventos y nodos confirmadores.
usuarios	nombre, email, teléfono, Telegram, rol, activo, fecha_registro, password_hash y archivos.
alertas	Eventos, usuarios, canal de envío, estado y fecha de notificación.
graficas_fft	Espectro FFT serializado asociado a una lectura.
eventos_hardware_sismo	timestamp_iso, fuente, puerto, estado, nivel, magnitud, RMS, aceleracion_max, payload JSON.

7. Estructura del Código

Carpeta/archivo	Descripción
server.js	Servidor principal, autenticación, CRUD, reportes, sismo en vivo y persistencia.
db.js	Conexión y helper de consultas MySQL.
uploads.js	Carga de archivos con Multer y validación de tipo/tamaño.
sismo_bridge.py	Lectura y procesamiento del sensor serial.
scripts/bootstrap-admin.js	Crea o actualiza el usuario administrador inicial.
arduino/sismografo_led_buzzer/sismografo_led_buzzer.ino	Firmware del sensor con LEDs y buzzer.
index.html	Interfaz pública con login, dashboard, modo educativo y acceso al administrador.
admin-panel.html	Panel administrativo con acceso, usuarios y explorador SQL.

public/reportes	Módulo de reportes y descarga de archivos.
uploads	Archivos subidos por usuario.

- Patrones/API usados: REST para la comunicación frontend-backend, JWT para autenticación, SSE para eventos en tiempo real, consultas parametrizadas en MySQL y static serving para HTML, reportes y archivos.
- Rutas principales: `/api/auth`, `/api/admin`, `/api/reportes`, `/api/sismo`, `/api/health`, `/reportes/` y `/uploads/`.
- Casos de uso principales del proyecto: registrar usuarios, iniciar sesión, administrar usuarios, generar reportes y simular lecturas sísmicas para la demo.

8. Pruebas Realizadas

Prueba	Resultado esperado / obtenido
GET /api/health	Responder con estado del servidor y conexión de base de datos.
POST /api/auth/register	Crear un usuario nuevo con hash de contraseña.
POST /api/auth/login	Generar token JWT válido para la sesión.
GET /api/admin/usuarios	Listar usuarios solo con rol admin.
GET /api/admin/db/tables	Listar tablas visibles para el explorador SQL.
POST /api/sismo/simular	Activar telemetría sintética para demo.
GET /api/sismo/live	Recibir flujo en tiempo real por SSE.
GET /api/reportes/usuarios/pdf	Generar el reporte PDF de usuarios.
GET /reportes/	Abrir el módulo de reportes con login, carga de archivos y descargas.

- Errores conocidos o comunes: MySQL apagado o credenciales incorrectas, token JWT vencido, intento de subir archivos no permitidos, puerto ocupado y ausencia del sensor físico en `COM3`.
- Cuando el sensor no está conectado, la demo se resuelve usando el modo de simulación web, sin bloquear la sustentación.

9. Conclusiones

- Dificultades encontradas: integración del puente serial, diferencias entre el modelo conceptual y el MVP implementado, y la dependencia del hardware para lecturas reales.
- Aprendizajes: uso de Express, JWT, MySQL, reportes PDF/Excel, almacenamiento de archivos, módulos educativos 3D y simulación de datos en tiempo real.
- Mejoras futuras: integrar el esquema distribuido completo, ampliar el modelo de alertas, conectar nodos físicos reales, mejorar los reportes y sumar analítica avanzada o IA.

10. Glosario

Término	Definición
MVP	Versión mínima funcional del sistema.
JWT	Token de autenticación firmado.
RMS	Medida de amplitud efectiva de la señal.
FFT	Transformada rápida de Fourier.
SSE	Server-Sent Events para streaming en tiempo real.
Multer	Middleware para subir archivos.
MySQL	Motor de base de datos relacional.
IoT	Dispositivos conectados que capturan y transmiten datos.
MPU6050	Sensor de aceleración y giro usado en el nodo sísmico.
MQTT	Protocolo ligero para intercambio de mensajes entre nodos y servidor.
TLS	Capa de seguridad para cifrar la comunicación.
Validación multi-nodo	Confirmación del evento cuando varios nodos coinciden.
Puente serial	Proceso que enlaza el sensor físico con la API web.
Three.js	Librería para gráficos 3D en navegador.

11. Referencias y recursos

- Repositorio de código fuente: el proyecto está en
`C:\Users\Administrator\Desktop\Desarrollo\_Web\mi\_proyecto`.
- Plantilla de manual técnico: `C:\Users\Administrator\Downloads\Plantilla\_Manual\_Tecnico.docx`.
- Documento completo del proyecto:
`C:\Users\Administrator\Downloads\SISMOGRAFO\_Proyecto\_Completo.docx`.
- Base SQL de referencia: `C:\Users\Administrator\Downloads\sismografo\_database.sql`.
- Guía de API y entrega: `C:\Users\Administrator\Desktop\Desarrollo\_Web\mi\_proyecto\API\_ENTREGA.md`.
- Documentación oficial: Node.js, Express, MySQL2, Multer, ExcelJS, PDFKit, Three.js, Chart.js, Leaflet y KaTeX.